Why we updated the original code

Project purpose: This project uses PointPillars to detect vehicles, pedestrians and cyclists in LiDAR point clouds, then publishes their 3D boxes through ROS 2 for visualization and use by other applications.

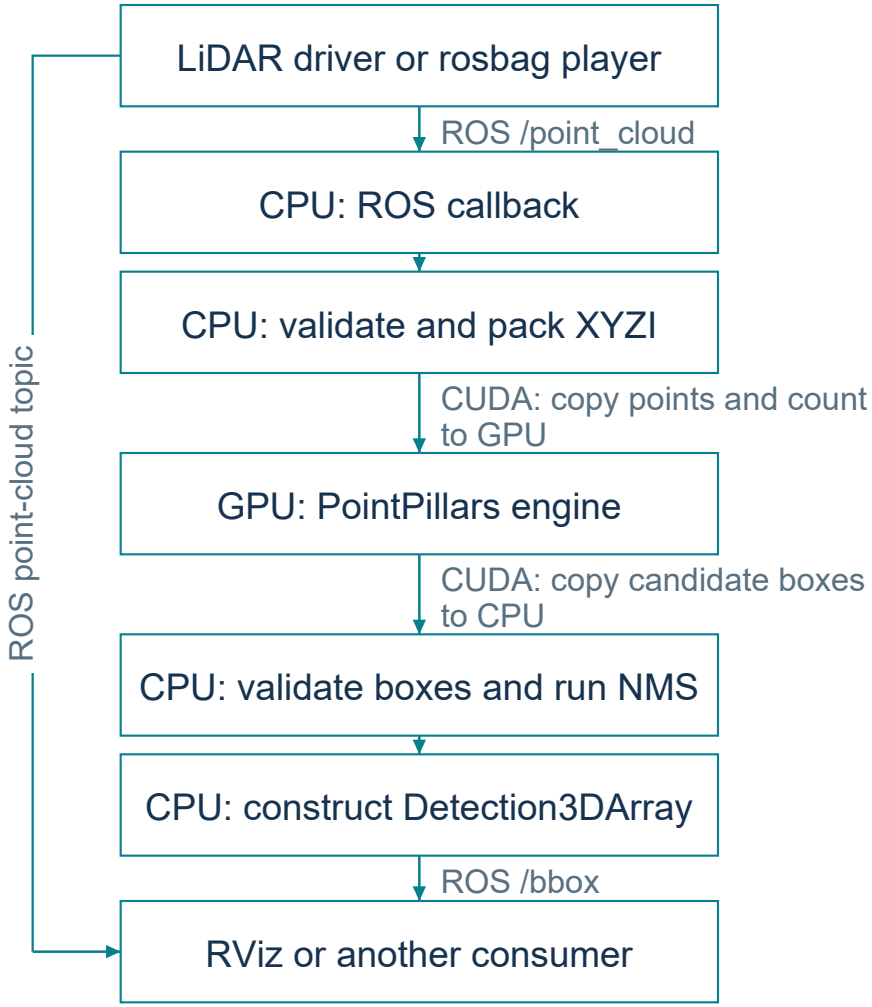


Same pretrained ONNX; no retraining.

Old ROS fields and TensorRT APIs did not fit the new stack. We also needed checked data sizes, reliable cleanup and fewer GPU allocations.

Main files we changed or added	What the change gives us
`CMakeLists.txt`, `package.xml`	C++17, explicit dependencies and CPU tests without a GPU.
`pp_inference_node.cpp`, `pointcloud_adapter.cpp`	Jazzy messages and checks before accepting a cloud.
`pointpillar.cpp`, `pointpillar.h`	TensorRT 10 named tensors, checked results and persistent runtime.
`cuda_resources.hpp`, `output_contract.hpp`	Reused GPU buffers, automatic cleanup and valid output counts.
`postprocess.cpp`, `postprocess.h`	Tested rotated box overlap and duplicate removal (NMS).

What the project supports today



During inference
TensorRT's CPU API schedules GPU work. ROS runs on the CPU.

CPU responsibility
Check and pack XYZI points. Validate returned boxes, remove duplicates with NMS, and publish detections with the cloud's timestamp and coordinate frame.

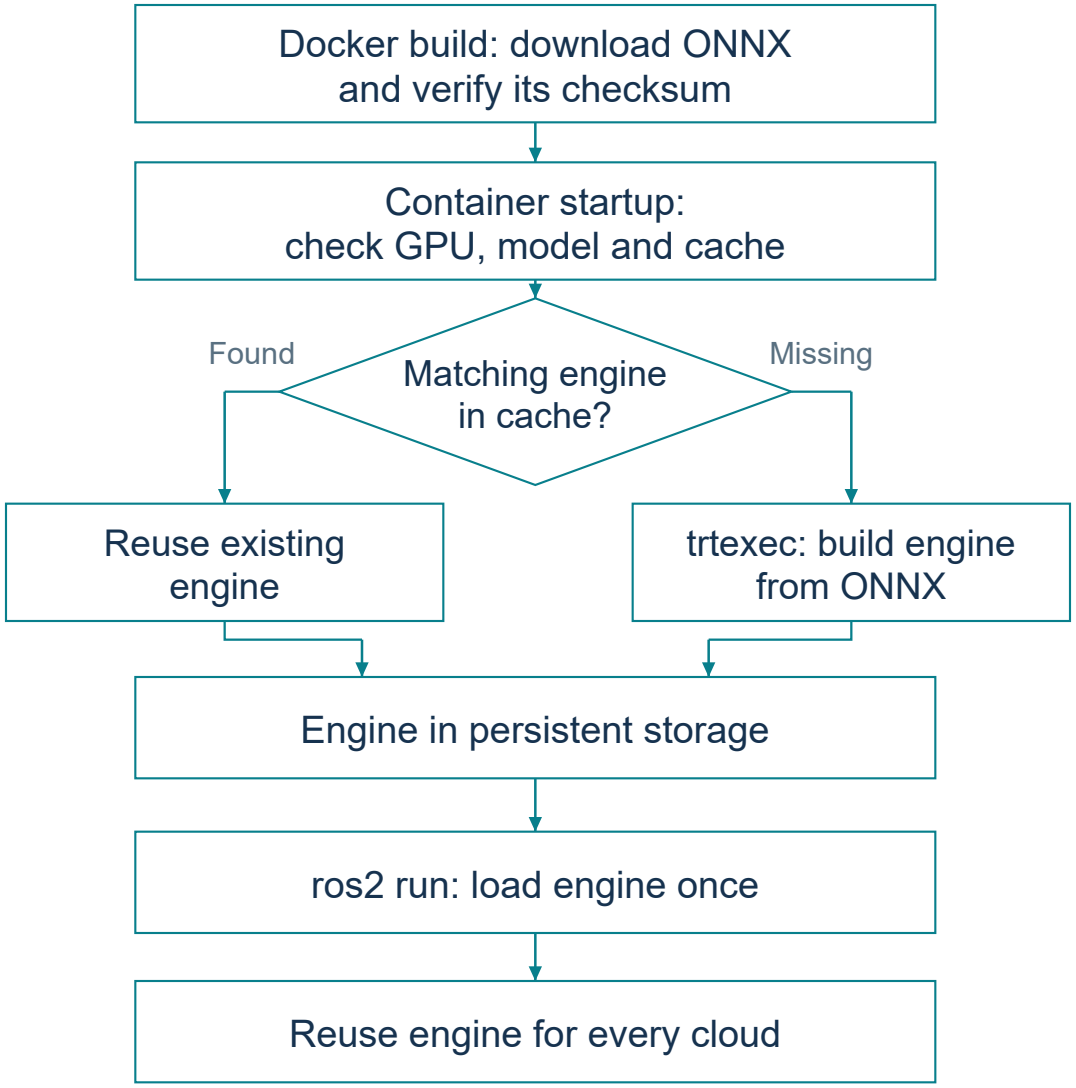
GPU responsibility
Group points into pillars, run the PointPillars network and decode candidate boxes.

Recorded checks
CPU/ROS tests, 10 synthetic GPU frames, engine-cache reuse and 25 bag frames in Docker plus 25 through host transport, with RViz.

Run settings
`deploy.sh` and `node.yaml`: build/run command and settings. The host provides the NVIDIA driver and Docker GPU access.

Tested: Ubuntu 24.04, Linux x86-64, RTX 4060 Laptop.

From ONNX to a running ROS node



Two files, two jobs
`.onnx`: trained network and weights, read by the builder. `.engine`: GPU execution plan, read by the ROS node. No retraining.

Where they are stored
Inside the container:
ONNX: `/opt/models/pointpillars.onnx`
Engine: `/var/lib/pp_infer/<key>.engine`
Default persistent volume: `pp-engines`.

When an engine is built
First startup uses `trtexec` to build an FP32 engine for the selected GPU. Later starts reuse a matching engine. The cache identity includes the model, GPU, driver and build settings.

What ROS loads
`engine_path` selects the engine. TensorRT loads it once into memory and prepares GPU buffers. Each cloud reuses it.

Keep the ONNX in the image: startup still checks its checksum. Direct `ros2 run` requires an already prepared engine.